{EPITECH}

SUMMER SCHOOL_

< PART 1 BOOTSTRAP - HTML & CSS FUNDAMENTALS />

SUMMER SCHOOL

< WEB DEVELOPMENT: THE BUILDING BLOCKS />

> ”The secret of getting ahead is getting started. The secret of getting started is breaking your complex overwhelming tasks into small manageable tasks, and starting on the first one.”
> — **Mark Twain**

Introduction

Before you build the visual prototype of the media portal, you need to understand the fundamental mechanics of the web. You cannot build a house without knowing how to lay bricks.

This bootstrap is a step-by-step playground. Your objective is not to build the final newspaper layout today, but to experiment with tags, stylesheets, and the rules of the web so you can approach the main project with confidence.

The Developer's Cheatsheet

Here is your arsenal. You do not need to memorize all of this immediately, but refer back to this list whenever you need to structure a new element or style it.

HTML Tags (The Skeleton)

Tag	Category	Purpose	Example Use Case
`<h1>` to `<h6>`	Text	Headings, from most to least important.	`<h1>Newspaper Title</h1>`
`<p>`	Text	A standard paragraph of text.	`<p>Breaking news today...</p>`
`<a>`	Link	An anchor to link to another page.	`<a href="page.html">Click here</a>`
`<img>`	Media	Embeds an image (self-closing).	`<img src="logo.png" alt="Logo">`
`<ul>` & `<ol>`	List	Unordered (bulleted) or Ordered (numbered) list.	`<ul>` wrapping `<li>` items.
`<li>`	List	A specific item inside a list.	`<li>First item</li>`
`<strong>`	Text	Bold text for strong importance.	`<strong>Warning!</strong>`
`<em>`	Text	Italic text for emphasis.	`<em>Really</em>` important.
`<div>`	Structure	A generic, invisible block container.	Grouping non-semantic elements.
`<span>`	Structure	A generic, invisible inline container.	Styling one word in a paragraph.
`<header>`	Semantic	The top section (usually logo and nav).	Wrapping the main navigation.
`<nav>`	Semantic	A specific block for navigation links.	Wrapping the menu items.
`<main>`	Semantic	The primary content of the page.	Wrapping the articles.
`<section>`	Semantic	A thematic grouping of content.	`<section id="politics-news">`
`<article>`	Semantic	An independent, self-contained story.	A single news card.
`<footer>`	Semantic	The bottom section (copyright, etc.).	Wrapping the bottom links.

CSS Properties (The Paint)

Property	Purpose	Example Values
`color`	Changes text color.	`red`, `#333333`, `rgb(255, 0, 0)`
`background-color`	Changes the background of a box.	`blue`, `#f4f4f9`, `transparent`
`font-size`	Adjusts the size of the text.	`16px`, `1.5rem`, `2em`
`font-weight`	Adjusts the thickness of the text.	`normal`, `bold`, `700`
`text-align`	Aligns text horizontally.	`left`, `center`, `right`, `justify`
`padding`	Space *inside* the box border.	`10px`, `20px 40px`
`margin`	Space *outside* the box border.	`15px`, `0 auto` (to center a block)
`border`	Adds a line around the box.	`1px solid black`
`border-radius`	Rounds the corners of the box.	`8px`, `50%` (for a circle)
`width` & `height`	Sets specific dimensions.	`100px`, `100%`, `50vw`
`display`	Changes how the box behaves.	`block`, `inline`, `flex`, `none`
`gap`	Space between items (requires Flexbox).	`20px`

Step 1: The Anatomy of HTML

HTML is a markup language. It uses **tags** to tell the browser what a piece of text is supposed to be.

The Golden Rules of HTML:

---

1. **Always close your tags:** Most tags come in pairs. An opening tag `<tag>` and a closing tag with a slash `</tag>`. If you forget the slash, the browser will break your layout.
2. **Indentation is mandatory:** Code is read by machines, but maintained by humans. When you place a tag inside another tag (nesting), you must press `Tab` to indent it. This makes the parent-child relationship visible.

Create an `index.html` file and type the following boilerplate:

```
<!DOCTYPE html>
<html>
<head>
    <title>My First Playground</title>
</head>
<body>
    <h1>Welcome to the Laboratory</h1>

    <div>
        <h2>Experiment 1</h2>
        <p>This paragraph is nested inside a division box.</p>
    </div>
</body>
</html>
```

Action: Open `index.html` in your browser. Notice how the `<h1>` is massive, the `<h2>` is slightly smaller, and the `<p>` is normal text.

If you see a **blank page**, check your tags. Did you forget a closing tag? Did you forget to save your file? Did you open the correct file in your browser?

Step 2: Semantic Containers

In the example above, we used a `<div>`. It is an invisible, generic box. However, modern web standards require **semantic tags** boxes that actually describe their purpose to search engines and screen readers.

Action: Replace the generic `<div>` in your code with an `<article>` tag, and add a link inside it using the `<a>` (anchor) tag.

```
<article>
    <h2>Campus Life Returns to Normal</h2>
    <p>After a busy week of exams, students are finally getting some rest.</p>
    <a href="https://www.epitech.eu">Read the full story</a>
</article>
```

Refresh your browser. Visually, nothing changed! Semantic tags behave exactly like `<div>` boxes, but they make your code professional.

Step 3: Introduction to CSS (The Paint)

HTML is just black and white text. To add styling, we use CSS.

Create a file named `styles.css` in the same folder. To connect your HTML to this new file, add the following line inside your `<head>` tags:

```
<link rel="stylesheet" href="styles.css">
```

Now, open `styles.css`. CSS uses **selectors** to target HTML tags, and applies rules using curly braces `{}`.

Action: Add these rules to your `styles.css` to test background colors and typography.

```
body {
    background-color: #f4f4f9; /* A very light grey */
    font-family: Arial, sans-serif; /* Changes the default font */
    color: #333333; /* Dark grey text is easier to read than pure black */
}

h1 {
    color: #2b6cb0; /* A nice blue */
    text-align: center; /* Centers the text horizontally */
}
```

Refresh your browser. Your page should now have a colored background, a new font, and a centered blue main title.

Step 4: The Box Model & Flexbox

This is the most critical concept in web development. Every element on your page is a rectangular box.
- **Padding:** The space *inside* the box (between the text and the border).
- **Margin:** The space *outside* the box (pushing other elements away).

Let's turn your `<article>` into a visible card. Add this to your CSS:

```
article {
    background-color: white;
    padding: 20px; /* Breathes space inside the card */
    margin: 20px; /* Pushes the card away from the edges of the screen */
    border-radius: 8px; /* Rounds the corners */
    box-shadow: 0px 4px 6px rgba(0, 0, 0, 0.1); /* Adds a subtle shadow */
}
```

Flexbox: The Layout Engine

By default, boxes stack vertically. If you want elements to sit side-by-side (like a navigation menu), you use Flexbox.

Action: Let's create a fake navigation menu. Add this to your HTML inside the `<body>`, right below your `<h1>`:

```
<nav class="menu">
    <a href="#">Home</a>
    <a href="#">Politics</a>
    <a href="#">Campus</a>
</nav>
```

Notice we added `class="menu"`. This allows us to target this specific `<nav>` in CSS using a dot `.`
Add this to your CSS:

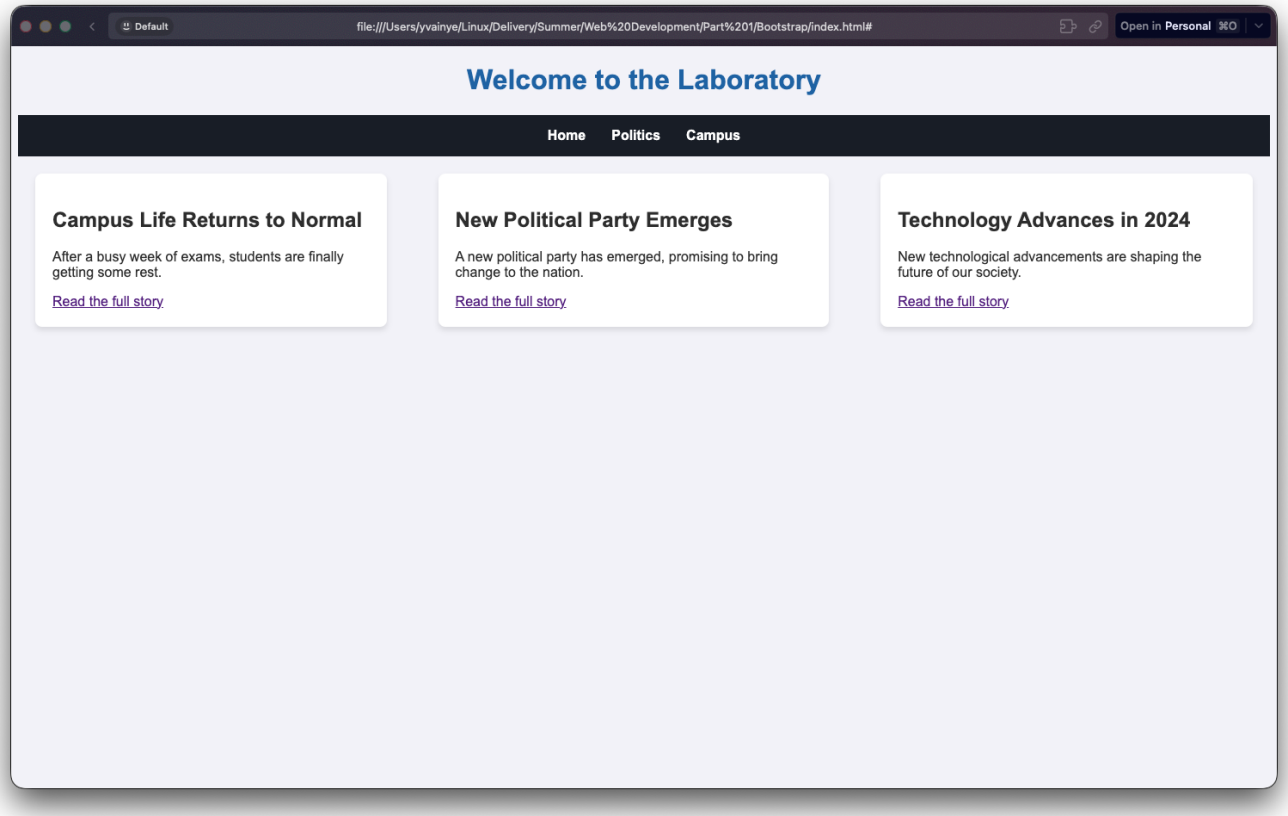
```
.menu {
    display: flex; /* Activates Flexbox */
    justify-content: center; /* Centers all items horizontally */
    gap: 30px; /* Adds space between the links */
    background-color: #1a202c;
    padding: 15px;
}

.menu a { /* Targets all <a> tags inside .menu */
    color: white; /* Makes the links white */
    text-decoration: none; /* Removes the default underline from links */
    font-weight: bold;
}
```

Step 5: The Final Assembly

You now have all the tools required to build the newspaper. You know how to:
1. Write clean, properly indented HTML.
2. Use semantic tags (`<article>`, `<nav>`).
3. Link a CSS file.
4. Manage colors and typography.
5. Create spacing using padding and margins.
6. Align elements using Flexbox.

Your Final Challenge: Before jumping into the main Project document, try to duplicate your `<article>` block in your HTML so you have three identical articles.
Wrap those three articles inside a `<section class="article-grid">` tag. Then, use `display: flex;` in your CSS to make the three articles sit perfectly side-by-side in a single row!



v1